UCS310 — Database Management System

Thapar Institute of Engineering and Technology

Complaint Management System

A Complete Oracle SQL & PL/SQL Database Project

Course	UCS310 — Database Management System
Institution	Thapar Institute of Engineering and Technology
Academic Year	2025–2026
Technology	Oracle SQL + PL/SQL | Oracle SQL Developer

Project Team	
Vinayak Agarwal	Roll No: 1024030915
Shubhr Gupta	Roll No: 1024030919
Aryan Juneja	Roll No: 1024030920

1. Project Overview

The Complaint Management System is a fully database-driven application built entirely with Oracle SQL and PL/SQL. It provides a structured, normalized relational database for logging, tracking, assigning, and resolving complaints raised by users across multiple departments.

In addition to core data management, the system incorporates advanced analytical capabilities using modern SQL techniques such as window functions and Common Table Expressions (CTEs). These enhancements enable performance analysis, ranking, trend detection, and decision-support insights directly within the database.

Key Objectives

- Store complainant, department, staff, and complaint data in a normalized schema.
- Automate status transitions and audit history using database triggers.
- Provide stored procedures for registering and assigning complaints.
- Perform advanced analytical reporting using subqueries, window functions, and CTEs.
- Enable ranking, benchmarking, and trend analysis directly within SQL queries.
- Demonstrate transaction control with SAVEPOINT, ROLLBACK, and COMMIT.

Technology Stack

Component	Details
Database Engine	Oracle Database
IDE / Tool	Oracle SQL Developer
Language	SQL and PL/SQL
Key Concepts	DDL, DML, DQL, Joins, Correlated Subqueries, EXISTS/NOT EXISTS, Window Functions (RANK, AVG OVER), Common Table Expressions (CTEs), Analytical CASE expressions, Views, Stored Procedures, Triggers, Cursors, Transactions

2. Database Schema

The system is built on ten fully normalized relational tables with appropriate primary keys, foreign key constraints, CHECK constraints, and sequences for auto-incrementing IDs. The schema follows third normal form (3NF) to eliminate data redundancy.

Table Name	Purpose
USERS	Complainant details — name, email, contact, registration date.
DEPARTMENT	Departments responsible for handling complaints.
STAFF	Staff members linked to departments, with contact details.
COMPLAINT_CATEGORY	Complaint types: Internet Issue, Power Outage, Cleanliness, etc.
COMPLAINT	Core table — description, status, priority, dates (submitted / resolved).
COMPLAINT_STATUS_HISTORY	Audit log of every status change with timestamps.
COMPLAINT_ASSIGNMENT	Maps complaints to staff members; tracks assignment date.
COMPLAINT_FEEDBACK	User ratings and comments for resolved complaints.
COMPLAINT_ATTACHMENT	File paths for evidence/attachments linked to complaints.
COMPLAINT_ESCALATION	Escalation level and escalation date per complaint.

Sample Data Volume

Entity	Rows Inserted
Users	30
Departments	8
Staff Members	20
Complaint Categories	10
Complaints	50
Status History Records	60
Assignments	40
Feedback Records	25
Attachments	20
Escalations	15

Complaint dates are distributed between 2024-01-01 and 2025-06-30, covering a realistic 18-month operational window.

3. Project File Structure

File	Description
01_create_tables.sql	DDL: Tables, constraints, sequences, and cleanup blocks.
02_insert_data.sql	DML: Realistic sample data insertion.
03_queries.sql	30 SELECT queries and CREATE VIEW statements.
04_plsql.sql	Procedures, functions, triggers, cursors, and transaction block.
complaint_management_system.sql	Combined single-file Oracle SQL + PL/SQL script.
README.md	Project overview, setup instructions, and execution guide.

Script Sections

Section	Content
SECTION 0	Cleanup: DROP tables/views/sequences before recreation.
SECTION 1	DDL: CREATE TABLE with primary keys, foreign keys, CHECK constraints.
SECTION 2	DML: INSERT INTO with 270+ realistic sample rows.
SECTION 3	SECTION 3: 30 advanced SELECT queries — correlated, nested, window-function based, and analytical queries.
SECTION 3A	CREATE VIEW for vw_complaint_summary, vw_department_performance, vw_staff_workload.
SECTION 4	PL/SQL: procedures, functions, triggers, and cursor blocks.
SECTION 5	Transaction: SAVEPOINT, ROLLBACK, and COMMIT demonstration.

4. SQL Query Coverage

The project contains **30 carefully designed SELECT queries** that demonstrate a combination of classical and modern SQL techniques. These include basic filtering, multi-table JOINs, aggregate functions with GROUP BY/HAVING, nested and correlated subqueries, and view-based analytical reporting.

Each query incorporates at least one advanced construct such as:

- Correlated and nested subqueries
- EXISTS / NOT EXISTS logic
- Window functions (RANK, AVG OVER)
- Common Table Expressions (CTEs)
- Analytical CASE expressions

Unlike traditional query sets focused on quantity, this project emphasizes **depth and real-world applicability**, solving analytical problems such as:

- Performance comparison across departments and staff
- Ranking and benchmarking
- Time-based trend analysis
- Identification of high-risk users and complaint patterns
- Dashboard-style multi-metric reporting

These queries collectively demonstrate SQL as both a **data retrieval and analytical tool**.

Range	Category	Description
Range	**Category**	**Description**
Q1 – Q8	Correlated Subqueries	Scalar subqueries in SELECT/WHERE/HAVING, per-row dynamic filtering
Q9 – Q16	EXISTS / NOT EXISTS	Semi-joins, anti-joins, relational division, logical filtering
Q17 – Q23	Nested & Multi-Level Queries	Deep nesting, inline views, multi-level aggregations
Q24 – Q27	Analytical & Conditional Queries	CASE expressions with subqueries, classification and comparisons
Q28 – Q30	Window Functions & Dashboard	RANK(), AVG OVER(), CTEs, performance metrics and dashboard-style reporting

Views Implemented

View Name	Description
vw_complaint_summary	Combines COMPLAINT, USERS, DEPARTMENT, and COMPLAINT_CATEGORY into one query-ready view.
vw_department_performance	Per-department: total complaints, resolved, pending, average rating, and avg resolution time.
vw_staff_workload	Per-staff: total assignments and resolved complaint count.

Some Queries(5 out of 30) and their results

1. **Relational Division Query**

```
SELECT u.user_id,
       u.name
FROM users u
WHERE NOT EXISTS (
   SELECT 1
   FROM department d
   WHERE EXISTS (
      SELECT 1
      FROM complaint_escalation e
      JOIN complaint ce
      ON e.complaint_id = ce.complaint_id

      WHERE ce.department_id = d.department_id
   )
   AND NOT EXISTS (
      SELECT 1
      FROM complaint c
      WHERE c.user_id = u.user_id
        AND c.department_id = d.department_id
   )
);
```

OUTPUT ➖

USER_ID	NAME
4	Aarav Sharma
11	Rohan Verma

2. Find complaint trends and rank months by complaint count.

```
SELECT TO_CHAR(c.date_registered, 'YYYY-MM') AS complaint_month,
       COUNT(*) AS total_complaints,
       RANK() OVER (
          ORDER BY COUNT(*) DESC
       ) AS month_rank
FROM complaint c
GROUP BY TO_CHAR(c.date_registered, 'YYYY-MM')
ORDER BY month_rank;
```

OUTPUT :-

COMPLAINT_MONTH	TOTAL_COMPLAINTS	MONTH_RANK
2025-03	8	1
2025-01	7	2
2024-11	6	3
2024-09	5	4

3. Find best-performing staff members in each department.

```
SELECT dept_name,
       staff_name,
       resolved_count
FROM (
   SELECT d.department_name AS dept_name,
          s.name AS staff_name,
          COUNT(c.complaint_id) AS resolved_count,
          RANK() OVER (
             PARTITION BY d.department_id
             ORDER BY COUNT(c.complaint_id) DESC
          ) AS dept_rank
   FROM department d
   JOIN staff s
   ON s.department_id = d.department_id
   LEFT JOIN complaint_assignment a
```

```
    ON s.staff_id = a.staff_id
    LEFT JOIN complaint c
    ON a.complaint_id = c.complaint_id
      AND c.status = 'Resolved'
    GROUP BY d.department_name,
             d.department_id,
             s.name
)
WHERE dept_rank = 1
ORDER BY dept_name;
```

OUTPUT :-

USER_ID	NAME	UNRESOLVED_COUNT	RISK_LEVEL
8	Ananya Iyer	5	High Risk
14	Rohan Verma	4	Medium Risk
21	Aarav Sharma	1	Low Risk

4. Classify users based on unresolved complaints.

```
    SELECT u.user_id,
u.name,
COUNT(c.complaint_id) AS unresolved_count,
CASE
  WHEN COUNT(c.complaint_id) >
    (
      SELECT AVG(cnt)
      FROM (
        SELECT COUNT(*) cnt
        FROM complaint
        WHERE status != 'Resolved'
        GROUP BY user_id
      )
    )
  THEN 'High Risk'
  WHEN COUNT(c.complaint_id) > 2
  THEN 'Medium Risk'
  ELSE 'Low Risk'
END AS risk_level
```

```
FROM users u
LEFT JOIN complaint c

ON u.user_id = c.user_id
WHERE c.status != 'Resolved'
GROUP BY u.user_id, u.name
ORDER BY unresolved_count DESC;
```

OUTPUT :-

USER_ID	NAME	UNRESOLVED_COUNT	RISK_LEVEL
8	Ananya Iyer	5	High Risk
14	Rohan Verma	4	Medium Risk
21	Aarav Sharma	1	Low Risk

5. Generate analytical dashboard for departments.

```
WITH dept_stats AS (
        SELECT d.department_id,
            d.department_name,
            COUNT(c.complaint_id) AS total_complaints,

            SUM(CASE WHEN c.status = 'Resolved' THEN 1 ELSE 0 END) AS resolved,
            AVG(
              CASE
                WHEN c.status = 'Resolved'
                THEN c.date_resolved - c.date_registered
              END
            ) AS avg_resolution_days,
            AVG(f.rating) AS avg_feedback_rating
        FROM department d
        LEFT JOIN complaint c
        ON d.department_id = c.department_id
        LEFT JOIN complaint_feedback f
        ON c.complaint_id = f.complaint_id
        GROUP BY d.department_id,
              d.department_name
    )
```

```
SELECT department_name,
    total_complaints,
    resolved,
    ROUND(avg_resolution_days, 2) AS avg_resolution_days,
    ROUND(avg_feedback_rating, 2) AS avg_feedback_rating,
    ROUND(
      (resolved * 100.0) /
      NULLIF(total_complaints, 0),
      2
    ) AS resolution_rate,
    RANK() OVER (
      ORDER BY resolved DESC
    ) AS performance_rank,

    CASE
      WHEN avg_resolution_days >
        AVG(avg_resolution_days) OVER ()
      THEN 'Below Avg'
      ELSE 'Above Avg'
    END AS efficiency
FROM dept_stats
ORDER BY performance_rank;
```

OUTPUT :-

DEPARTMENT_NAME	TOTAL_COMPLAINTS	RESOLVED	AVG_RESOLUTION_DAYS	RESOLUTION_RATE	PERFORMANCE_RANK	EFFICIENCY
IT	12	9	3.11	75.00	1	Above Avg
Maintenance	10	7	4.85	70.00	2	Below Avg
Security	8	5	5.22	62.50	3	Below Avg
Electrical	7	5	3.90	71.43	4	Above Avg

5. PL/SQL Features

All required PL/SQL components have been implemented in **04_plsql.sql** (SECTION 4) of the project. The table below provides a complete verification summary. Every component includes proper exception handling using EXCEPTION / WHEN OTHERS blocks.

Type	Name	Description
Procedure	`register_complaint`	Registers a new complaint row in COMPLAINT table and inserts the initial 'Submitted' record into COMPLAINT_STATUS_HISTORY. Accepts user_id, category_id, department_id, description, and priority as IN parameters.
Procedure	`assign_complaint`	Assigns a complaint to a specified staff member in COMPLAINT_ASSIGNMENT. Automatically updates complaint status to 'In Progress'. Includes exception handling for invalid IDs.
Function	`get_resolved_count`	Accepts a department_id as IN parameter. Queries COMPLAINT table and returns the INTEGER count of complaints with status = 'Resolved' for that department.
Function	`get_avg_rating`	Accepts a department_id. Joins COMPLAINT and COMPLAINT_FEEDBACK, returns the FLOAT average feedback rating for resolved complaints in that department.
Trigger	`trg_auto_resolve_date`	BEFORE UPDATE trigger on COMPLAINT. Fires when the status column changes to 'Resolved'. Automatically sets DATE_RESOLVED = SYSDATE so no manual date entry is required.
Trigger	`trg_status_history`	AFTER UPDATE trigger on COMPLAINT. Fires whenever the STATUS column value changes and inserts a new row into COMPLAINT_STATUS_HISTORY to maintain a complete audit trail.
Cursor Block	`Department Report Cursor`	Explicit cursor iterating over DEPARTMENT. For each department, fetches total complaints, resolved complaints, and pending complaints. Outputs formatted report via DBMS_OUTPUT.PUT_LINE.
Cursor Block	`Escalation Report Cursor`	Explicit cursor joining COMPLAINT_ESCALATION, COMPLAINT, USERS, and DEPARTMENT. Prints escalation level, user name, and department for all escalated complaints.

Transaction Management (SECTION 5)

A dedicated transaction block demonstrates Oracle's save point mechanism. The block inserts a complaint, assigns it to staff, records status history, then intentionally rolls back to a savepoint to demonstrate partial rollback, followed by a final COMMIT. This verifies atomicity and transaction control in a realistic multi-step workflow.

- SAVEPOINT sp1 — checkpoint before first insert
- INSERT into COMPLAINT — new complaint row
- SAVEPOINT sp2 — checkpoint after complaint insert
- INSERT into COMPLAINT_ASSIGNMENT — assign to staff
- INSERT into COMPLAINT_STATUS_HISTORY — initial status
- ROLLBACK TO sp2 — undo assignment while keeping complaint
- COMMIT — persist the complaint row permanent

6. PL/SQL

Registering a Complaint

```
SET SERVEROUTPUT ON;
```

```
BEGIN
register_complaint(
p_user_id => 1,
p_category_id => 4,
p_department_id => 2,
p_description => 'New Wi-Fi issue reported from library.',
p_priority => 'High'
);
END;
/
```

Assigning a Complaint to Staff

```
BEGIN
assign_complaint(
p_complaint_id => 1001,
p_staff_id => 3
);
END;
/
```

Calling Functions

```
-- Returns resolved complaint count for department 2
SELECT get_resolved_count(2) AS resolved_it_complaints FROM dual;
-- Returns average feedback rating for department 2
SELECT get_avg_rating(2) AS avg_it_rating FROM dual;
```

Trigger: Auto-Set Resolved Date

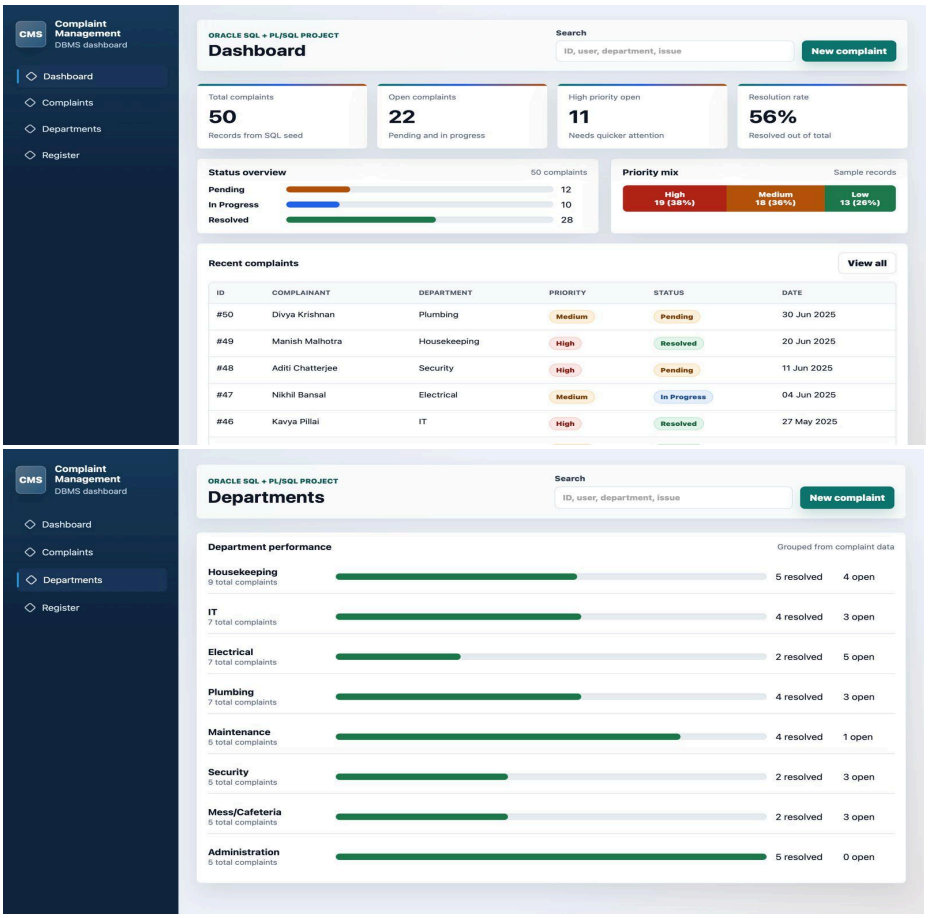
```
CREATE OR REPLACE TRIGGER trg_auto_resolve_date
BEFORE UPDATE ON COMPLAINT
FOR EACH ROW
BEGIN
IF :NEW.STATUS = 'Resolved' AND :OLD.STATUS != 'Resolved' THEN
:NEW.DATE_RESOLVED := SYSDATE;
END IF;
END;
```

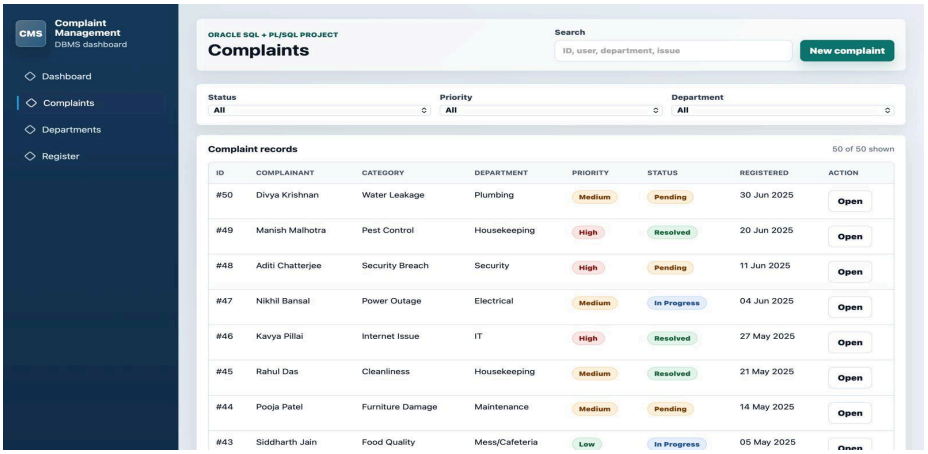
```
/
```

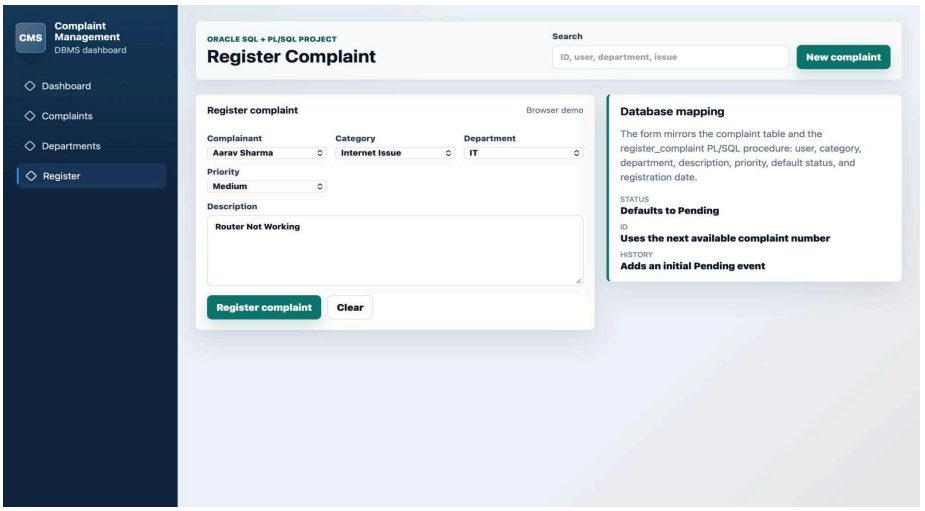
Cursor Block: Department Report

```
DECLARE
CURSOR dept_cur IS
SELECT D.DEPT_NAME,
COUNT(C.COMPLAINT_ID) AS total,
SUM(CASE WHEN C.STATUS='Resolved' THEN 1 ELSE 0 END) AS resolved,
SUM(CASE WHEN C.STATUS!='Resolved' THEN 1 ELSE 0 END) AS pending
FROM DEPARTMENT D
LEFT JOIN COMPLAINT C ON D.DEPT_ID = C.DEPT_ID
GROUP BY D.DEPT_NAME;
BEGIN
FOR rec IN dept_cur LOOP
DBMS_OUTPUT.PUT_LINE('Dept: ' || rec.DEPT_NAME ||
' | Total: ' || rec.total ||
' | Resolved: ' || rec.resolved ||
' | Pending: ' || rec.pending);
END LOOP;
END;
/
```

7. Dashboard/ Front End

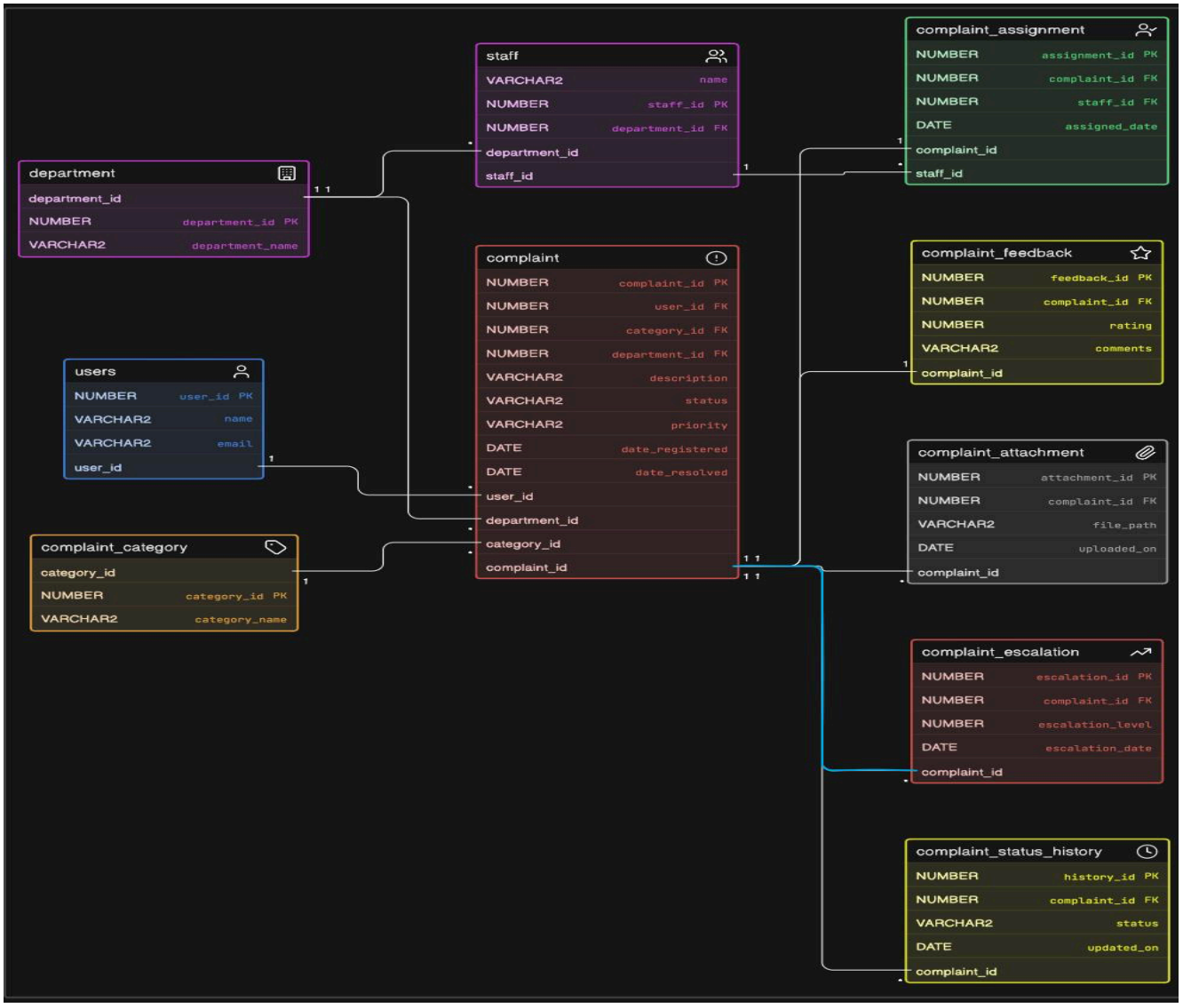



CMS
Complaint Management
DBMS dashboard
Dashboard
Complaints
Departments
Register
ORACLE SQL + PL/SQL PROJECT
Complaints
Search
ID, user, department, issue
New complaint
Status
All
Priority
All
Department
All
Complaint records
50 of 50 shown
ID COMPLAINANT CATEGORY DEPARTMENT PRIORITY STATUS REGISTERED ACTION
#50 Divya Krishnan Water Leakage Plumbing Medium Pending 30 Jun 2025 Open
#49 Manish Malhotra Pest Control Housekeeping High Resolved 20 Jun 2025 Open
#48 Aditi Chatterjee Security Breach Security High Pending 11 Jun 2025 Open
#47 Nikhil Bansal Power Outage Electrical Medium In Progress 04 Jun 2025 Open
#46 Kavya Pillai Internet Issue IT High Resolved 27 May 2025 Open
#45 Rahul Das Cleanliness Housekeeping Medium Resolved 21 May 2025 Open
#44 Pooja Patel Furniture Damage Maintenance Medium Pending 14 May 2025 Open
#43 Siddharth Jain Food Quality Mess/Cafeteria Low In Progress 05 May 2025 Open


CMS
Complaint Management
DBMS dashboard
Dashboard
Complaints
Departments
Register
ORACLE SQL + PL/SQL PROJECT
Register Complaint
Search
ID, user, department, issue
New complaint
Register complaint
Browser demo
Complainant
Aarav Sharma
Category
Internet Issue
Department
IT
Priority
Medium
Description
Router Not Working
Register complaint
Clear
Database mapping
The form mirrors the complaint table and the register_complaint PL/SQL procedure: user, category, department, description, priority, default status, and registration date.
STATUS
Defaults to Pending
ID
Uses the next available complaint number
HISTORY
Adds an initial Pending event

8. Entity Relation Diagram(ER)

The Entity Relationship (ER) Diagram illustrates the overall database structure of the Complaint Management System and defines the relationships between all major entities such as Users, Complaints, Departments, Staff, and Feedback,etc. It provides a clear representation of primary keys, foreign keys, and table associations to ensure efficient data organization, integrity, and smooth system operations.



9. Execution Guide

The project requires Oracle Database with Oracle SQL Developer. No other software is needed. All scripts are standard Oracle SQL/PL/SQL and use no third-party packages or extensions.

Step-by-Step Instructions

Step	Action
Step 1	Install Oracle Database and Oracle SQL Developer on your machine.
Step 2	Open Oracle SQL Developer and connect to your Oracle schema/user.
Step 3	Open and run 01_create_tables.sql — creates all tables and sequences.
Step 4	Open and run 02_insert_data.sql — inserts all sample data.
Step 5	Open and run 03_queries.sql — executes 30 SELECT queries and creates views.
Step 6	Enable DBMS_OUTPUT: run SET SERVEROUTPUT ON in a SQL Worksheet.
Step 7	Open and run 04_plsql.sql — compiles procedures, functions, triggers, and cursor blocks.
Step 8	View output in the Script Output panel at the bottom of SQL Developer.

10. Project Summary & Concepts Covered

DDL	CREATE TABLE, DROP, ALTER, constraints (PK, FK, CHECK, NOT NULL), SEQUENCE
DML	INSERT INTO with 270+ rows of realistic data across 10 tables
DQL	30 SELECT queries: basic, join, aggregate, subquery, analytical
Views	3 analytical views encapsulating complex multi-table logic
Procedures	register_complaint, assign_complaint — parameterized, with exceptions
Functions	get_resolved_count, get_avg_rating — return scalar values to caller
Triggers	trg_auto_resolve_date (BEFORE), trg_status_history (AFTER)
Cursors	2 explicit cursor blocks with FOR loop and DBMS_OUTPUT reporting
Transactions	SAVEPOINT, ROLLBACK TO savepoint, COMMIT in a multi-step block
Exception Handling	EXCEPTION / WHEN OTHERS in all PL/SQL units